

ODSC AI

The #1 AI Builders Conference

AI FOR SOFTWARE DEVELOPERS

SAN FRANCISCO | OCT 27-29

INTRODUCTION TO THE MATH BEHIND TRANSFORMERS AND LLMS

DAVID HOYLE

Research Data Science Specialist

dunnhumby

github.com/dchoyle/ODSCWest2026_TransformerBootcamp

Outline

01	What we will learn
02	Predicting the next word
03	Simple approaches
04	Word embeddings
05	Transformers
06	Back to predicting the next word
07	Different types of transformers
08	Wrap-up

github.com/dchoyle/ODSCWest2026_TransformerBootcamp

Outline

01	What we will learn
02	Predicting the next word
03	Simple approaches
04	Word embeddings
05	Transformers
06	Back to predicting the next word
07	Different types of transformers
08	Wrap-up





Break

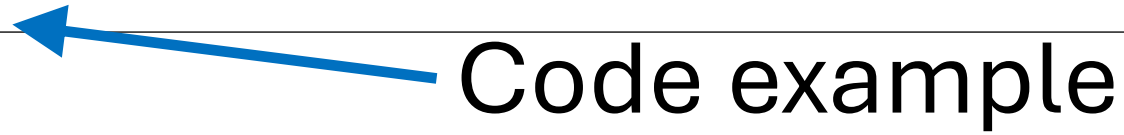
github.com/dchoyle/ODSCWest2026_TransformerBootcamp

Outline

01	What we will learn
02	Predicting the next word
03	Simple approaches
04	Word embeddings
05	Transformers
06	Back to predicting the next word
07	Different types of transformers
08	Wrap-up







What we will learn

- The mathematical challenges of next word prediction
- **The basic mathematical ideas behind Transformers**
- How Transformers predict words and sentences
- **To get comfortable at a high-level with how LLMs work at a mathematical level**
- To lose your fear of the science of LLMs



What we will **not** learn

- How to train foundational models
- How to build transformer-based chatbots
- How to prompt
- Agentic AI
- How to make $$$Millions from AI



What I assume you know

- Basic linear algebra, e.g. matrix multiplication as a transformation
- Basic machine learning, e.g. training a model by minimizing a loss function



Predicting the next word

What this enables, tokens and sequences

Predicting the next word

- What are we trying to achieve?
    - The cat sat on …….

Predicting the next word

- What are we trying to achieve?
  - The cat sat on the
  - The cat sat on the mat

Words as tokens

- Words are the natural units of a sentence.
- We call these units “tokens”
- Our modelling challenge is “predict the next token in given the preceding sequence of tokens”
- Tokens don’t have to be complete words:

  quick brown fox jumped over → [quick, brown, fox, jump, ed, over]

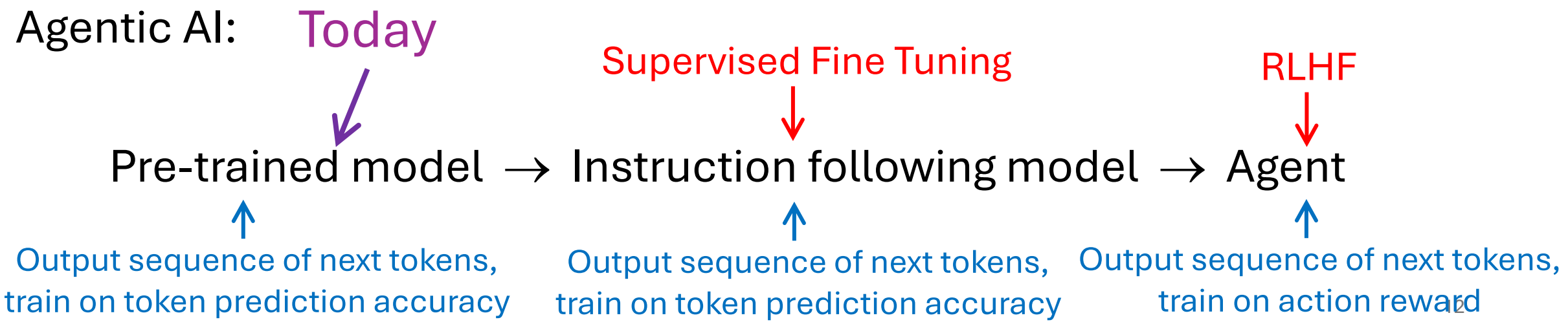
Sequences

- We find sequences of tokens everywhere, not just in language

- Sequence can be structured code, including tool calls

```
{“tool_call”:{ “tool_name”: “flight_booking_api”, “arguments”: {
      “destination”: “LHR”, “max_price”: 400, “seat_preference”: “window”}
      ...}
```

- Agentic AI:



Simple approaches

Naïve models that are wrong and what we can learn from that

Simple approach:

Example: “The cat sat on the mat”

5 words in the vocabulary ⇒ represent each word as 5-dimensional vector

“the” = (1,0,0,0,0)
“cat” = (0,1,0,0,0)
“sat” = (0,0,1,0,0)
“on” = (0,0,0,1,0)
“mat” = (0,0,0,0,1)

Simple approach:

Prompt: “The cat sat on the”

We could represent the 5-word initial prompt as combination of the individual word vectors

Prompt =

the	cat	sat	on	the
1	0	0	0	1
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

Input prompt space is now $5^5 = 3125$ dimensional

Simple approach: Issues

Problems:

1. Starting dimensionality is high

   Our vocabulary is large, e.g. 170000 – 700000 words

2. We need to work with long contexts → large prompt representation

Solution:

a) Reduce the dimensionality ← Use word embedding vectors

b) Represent words using vectors that incorporate the preceding context ← Create context-aware word embedding vectors

Word embeddings

Reducing the dimensionality

Word embeddings

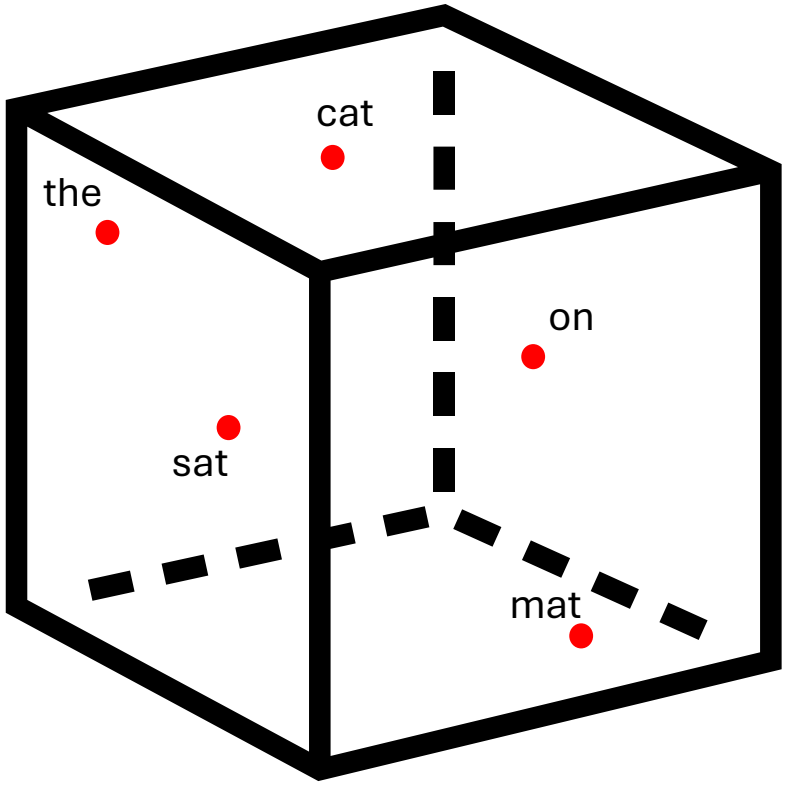
- Reduce to a fixed lower dimensionality

5-dimensional

"the" = (1,0,0,0,0)
"cat" = (0,1,0,0,0)
"sat" = (0,0,1,0,0)
"on" = (0,0,0,1,0)
"mat" = (0,0,0,0,1)

Map

3-dimensional



Word embeddings

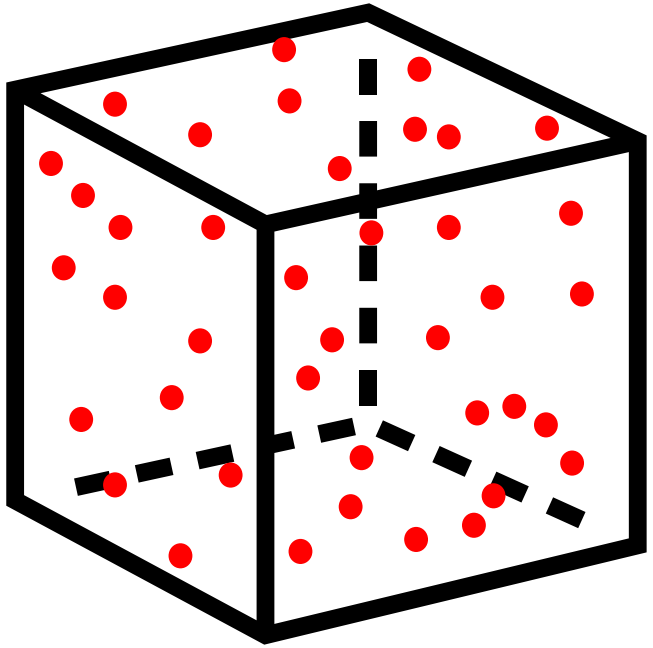
- Reduce to a fixed lower dimensionality

Word2Vec: 300-dimensional
GPT-3 (Davinci): 12288-dimensional
LlaMA 3.1-70B: 8192-dimensional
GPT-4.5: 3072-dimensional

170000 – 700000 dimensional

All of English language

Map



Word embeddings

Tokens →

Index	**the**	**cat**	**sat**	**on**	**the**	**mat**
1	2.4249×10^{-2}	4.7426×10^{-2}	-8.1093×10^{-2}	-2.0204×10^{-2}	2.4249×10^{-2}	-7.3271×10^{-2}
2	4.8235×10^{-3}	-4.2203×10^{-2}	-6.3660×10^{-2}	-6.6599×10^{-2}	4.8235×10^{-3}	-8.9358×10^{-2}
3	1.8411×10^{-2}	2.8491×10^{-2}	3.6398×10^{-2}	2.5621×10^{-2}	1.8411×10^{-2}	1.0698×10^{-1}
4	1.1867×10^{-2}	-4.4481×10^{-2}	3.3358×10^{-2}	1.7926×10^{-2}	1.1867×10^{-2}	1.1881×10^{-1}
5	1.9167×10^{-2}	-2.6467×10^{-2}	1.1537×10^{-1}	8.4262×10^{-23}	1.9167×10^{-2}	5.6133×10^{-3}
⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮
300	3.4856×10^{-3}	1.3847×10^{-1}	9.3088×10^{-2}	5.2301×10^{-3}		1.8160×10^{-1}

↓ Embedding vector

dimension d

Word2Vec model = “fasttext-wiki-news-subwords-300”

Code example 1

Working with word embeddings

Code Example 1

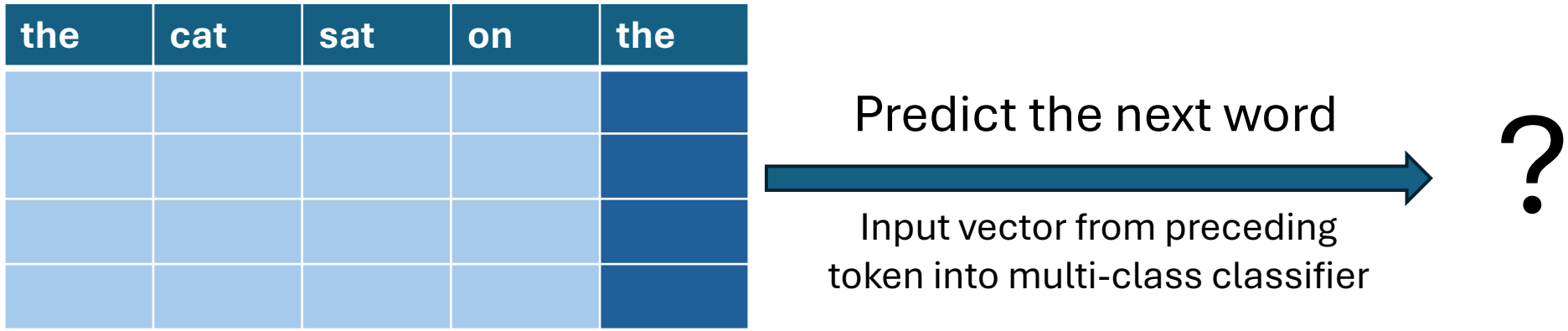
- Open up the Jupyter notebook ODSCWest2026_CodeExample1.ipynb in the github repository

https://github.com/dchoyle/ODSCWest2026_TransformerBootcamp/

Transformers

Creating context aware embeddings

Transforming embeddings



- Each vector knows nothing about the broader context of the preceding words
- Need new vectors for each token that take into account the preceding context
  - We will make a Transformer
- For now, create new vectors that take into account the context of the whole sentence

Attention

Input embeddings

the	cat	sat	on	the



Updated embeddings

the	cat	sat	on	the

- New vector for token n = weighted average across all vectors in sentence

$$\underline{y}_n = \sum_{n'} \underline{y}_{n'} A_{n,n'} \quad , \quad \sum_{n'} A_{n,n'} = 1 \;\; \forall n$$

- Actually, let's take weighted average of vectors that we'll learn as part of the training process

$$\underline{y}_n = \sum_{n'} \underline{v}_{n'} A_{n,n'} \quad , \quad \underline{v}_n^{\mathsf{T}} = \underline{y}_n^{\mathsf{T}} \underline{U}_V$$

Self-Attention

- Where do we get $A_{n,n'}$ from?
  1) Needs to depend upon vectors for tokens n and n'
  2) Needs to reflect similarity between vectors for tokens n and n'

$$\Rightarrow A_{n,n'} \sim \underline{y}_n \cdot \underline{y}_{n'} = \underline{y}_n^{\top} \underline{y}_{n'}$$

Not a probability !

- Not a problem – exponentiate and normalize

$$A_{n,n'} = \frac{\exp\left(\underline{y}_n \cdot \underline{y}_{n'}\right)}{\Sigma_{n''} \exp\left(\underline{y}_n \cdot \underline{y}_{n''}\right)}$$

just softmax function applied to $\underline{y}_n \cdot \underline{y}_{n'}$

Self-Attention

- Are $\underline{y}_n$ and $\underline{y}_{n'}$ the best vectors to measure similarity between tokens n and n' ?

- Ok, let's allow for a learnable linear transformation

$$\underline{y}_n \rightarrow \underline{\underline{U}}\,\underline{y}_n = \underline{z}_n$$

- Similarity between tokens is now defined as,

$$\underline{z}_n \cdot \underline{z}_{n'} = \underline{y}_n^{\top}\,\underline{\underline{U}}^{\top}\,\underline{\underline{U}}\,\underline{y}_{n'}$$

- Introduce more flexibility: 2 transformation matrices

$$\underline{z}_n^{(q)} \cdot \underline{z}_{n'}^{(k)} = \underline{y}_n^{\top}\underline{\underline{U}}_q^{\top}\underline{\underline{U}}_k\underline{y}_{n'} \qquad \underline{\underline{U}}_q\,,\underline{\underline{U}}_k = d \times d_k\,,\; d_k < \mathrm{d}$$

- Similarity between token n and n' now given by

$$\underline{y}_n^{\top}\underline{\underline{U}}_q^{\top}\underline{\underline{U}}_k\underline{y}_{n'}$$

Self-Attention

- Are $\underline{y}_n$ and $\underline{y}_{n'}$ the best vectors to measure similarity between tokens n and n' ?

- Ok, let's allow for a learnable linear transformation

$$\underline{y}_n \rightarrow \underline{\underline{U}}\,\underline{y}_n = \underline{z}_n$$

- Similarity between tokens is now defined as,

$$\underline{z}_n \cdot \underline{z}_{n'} = \underline{y}_n^{\top}\,\underline{\underline{U}}^{\top}\,\underline{\underline{U}}\,\underline{y}_{n'}$$

- Introduce more flexibility: 2 transformation matrices

$$\underline{z}_n^{(q)} \cdot \underline{z}_{n'}^{(k)} = \underline{y}_n^{\top}\underline{\underline{U}}_q^{\top}\underline{\underline{U}}_k\underline{y}_{n'} \qquad \underline{\underline{U}}_q\,,\underline{\underline{U}}_k = d \times d_k\,,\; d_k < \mathrm{d}$$

- Similarity between token n and n' now given by

$$\underline{y}_n^{\top}\underline{\underline{U}}_q^{\top}\underline{\underline{U}}_k\underline{y}_{n'}/\sqrt{d_k}$$

Self-Attention

- Putting it altogether (almost)

$$\text{New embedding vector for } n = \sum_{n'} v_{n'} \frac{\exp\left(\underline{y}_n^\top \underline{\underline{U}}_q^\top \underline{\underline{U}}_k \underline{y}_{n'}\right)}{\sum_{n''} \exp\left(\underline{y}_n^\top \underline{\underline{U}}_q^\top \underline{\underline{U}}_k \underline{y}_{n''}\right)}$$

- In English:

  New embedding vector for token n = weighted average of the vectors $\underline{v}_{n'}$

  $\text{weight}(n, n')$ = Softmax(similarity between transformed vectors for n and n')

  We learn the vectors $\underline{v}_n$ and the projection matrices $\underline{\underline{U}}_q$, $\underline{\underline{U}}_k$ through training

Attention: Masking

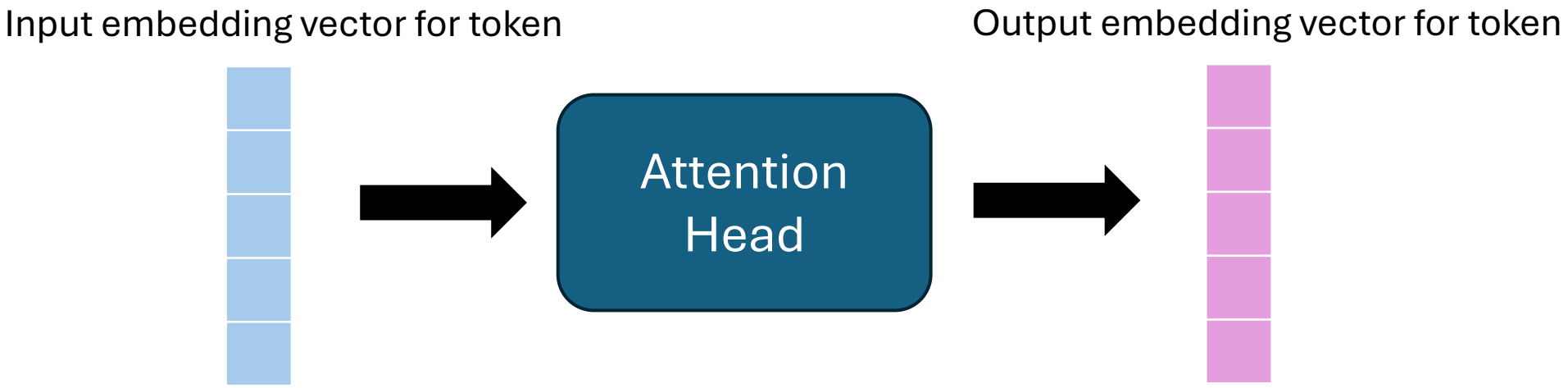
- When we are building a model to predict the next word we only want to take into account the preceding context

- We want $A_{n,n'} = 0$ if $n' > n$

	the	cat	sat	on	the
the	1.0	0	0	0	0
cat	0.2	0.8	0	0	0
sat	0.35	0.15	0.5	0	0
on	0.08	0.12	0.34	0.46	0
the	0.2	0.25	0.30	0.05	0.2

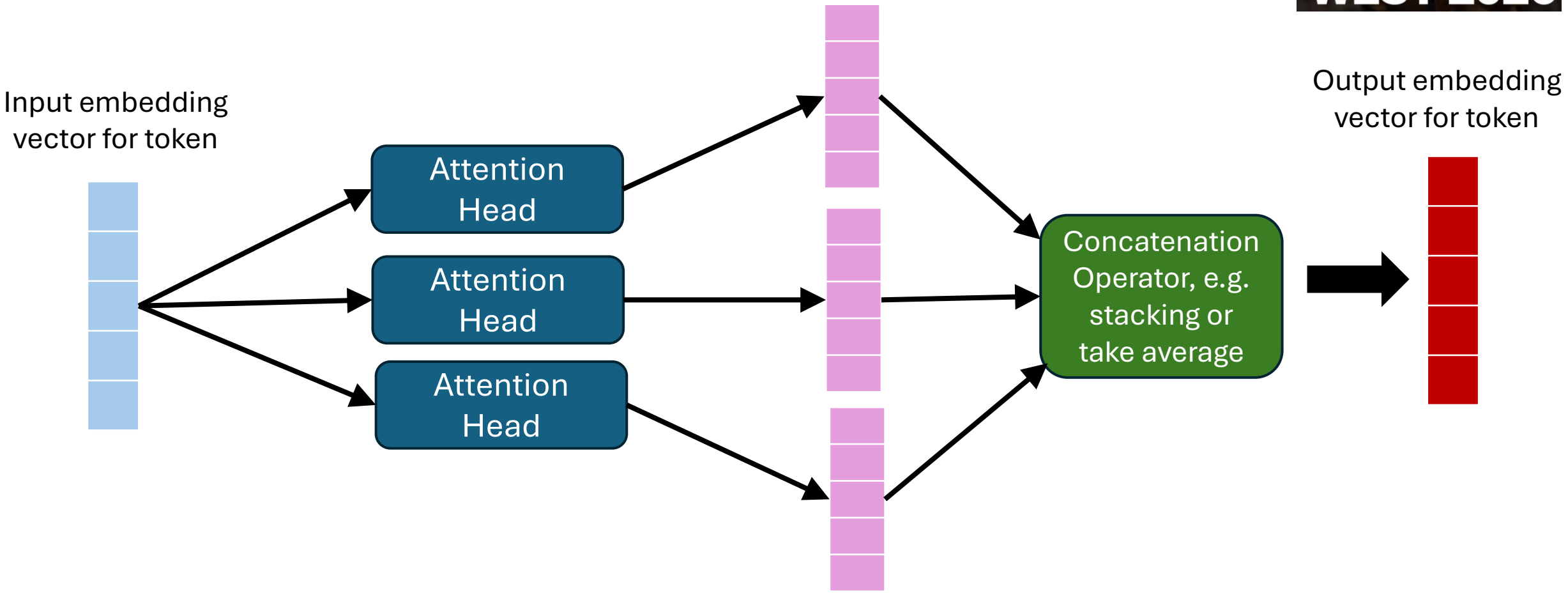
- We can easily achieve this by putting correspond ‘similarity’ values equal to $-\infty$ before applying softmax operation

Attention Head



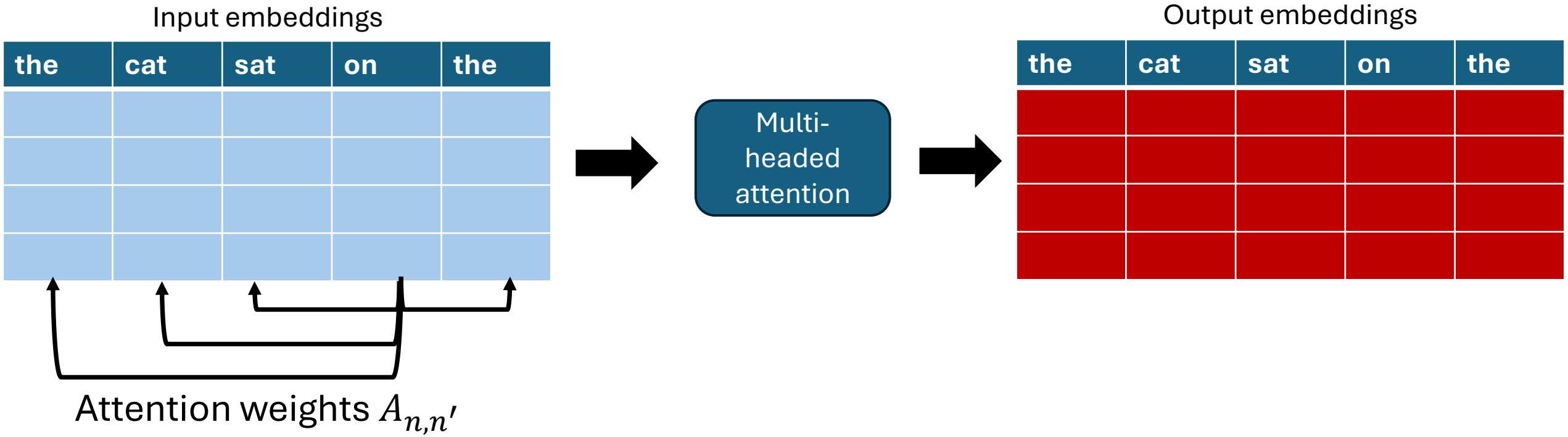
- We can wrap all the mathematics of self-attention into an operation we will call an Attention Head
- An attention head maps an embedding vector to an updated embedding vector taking into account what the surrounding words (tokens) are
- Problem: We use the same attention weight $A_{n,n'}$ for each dimension of the embedding space
  We can't have two tokens similar in one dimension and very dis-similar in another dimension

Multi-headed Self-Attention



- We solve this by having multiple (H) self-attention heads.
- We combine the output vectors of all the attention heads into a single vector
- Typically, we make output dimension of each attention head d/H

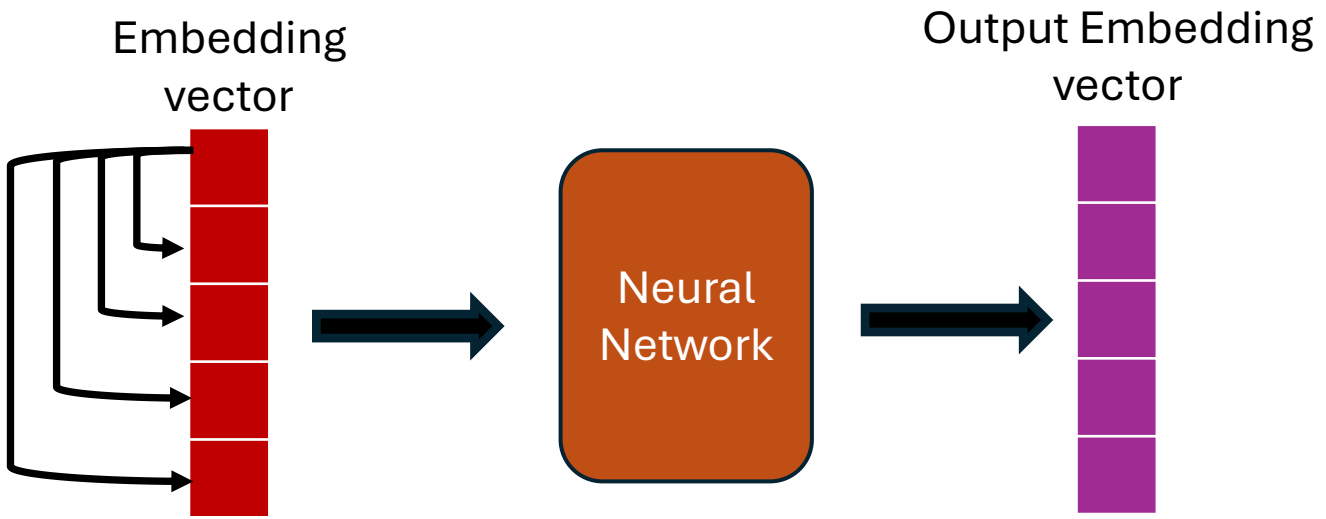
Neural Network Sub-Layer



- Attention process is column-wise (token-to-token)

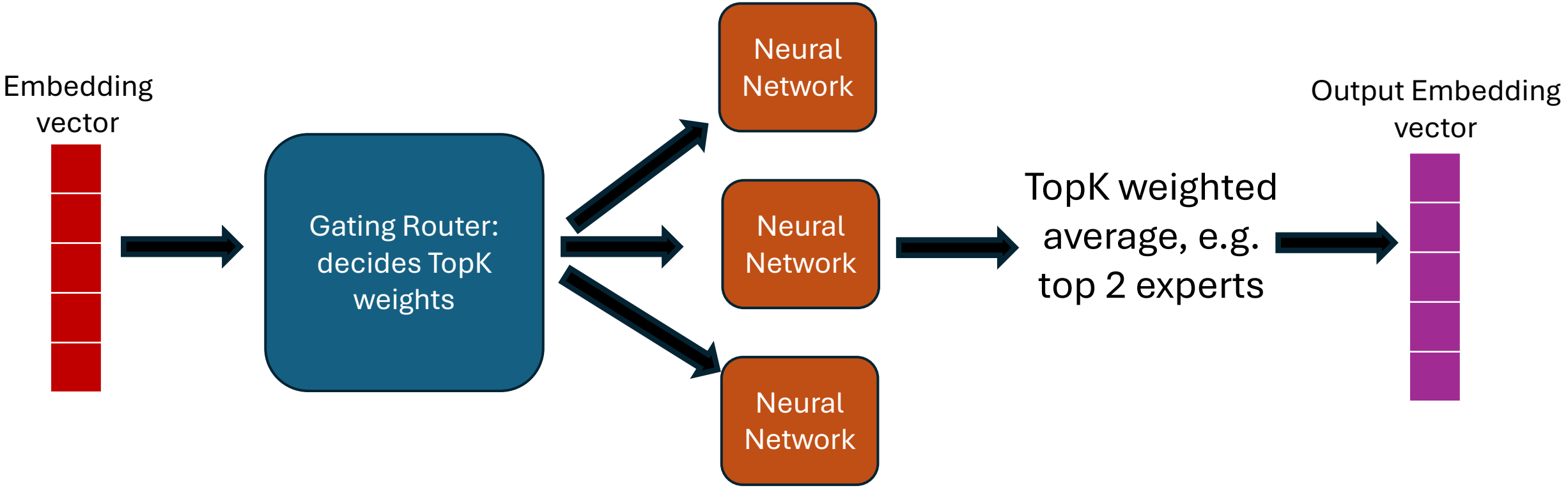
- We haven't allowed the different rows within a column to interact when producing the updated embedding

Neural Network Sub-Layer

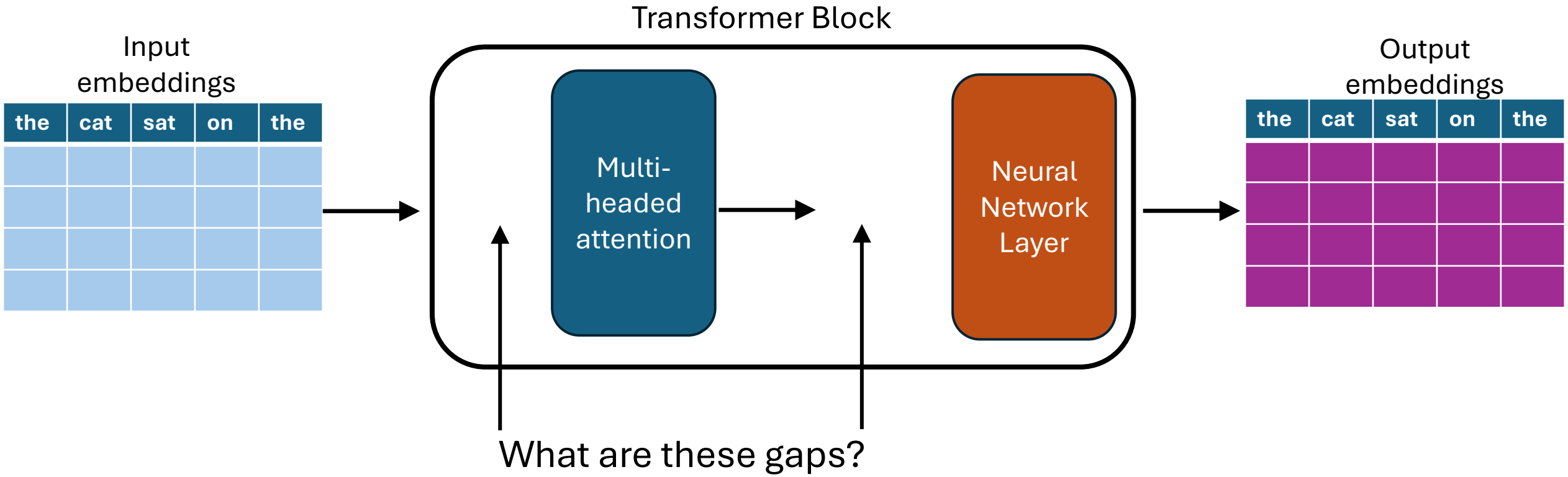


Parameters of neural network are learnt as part of the overall model training

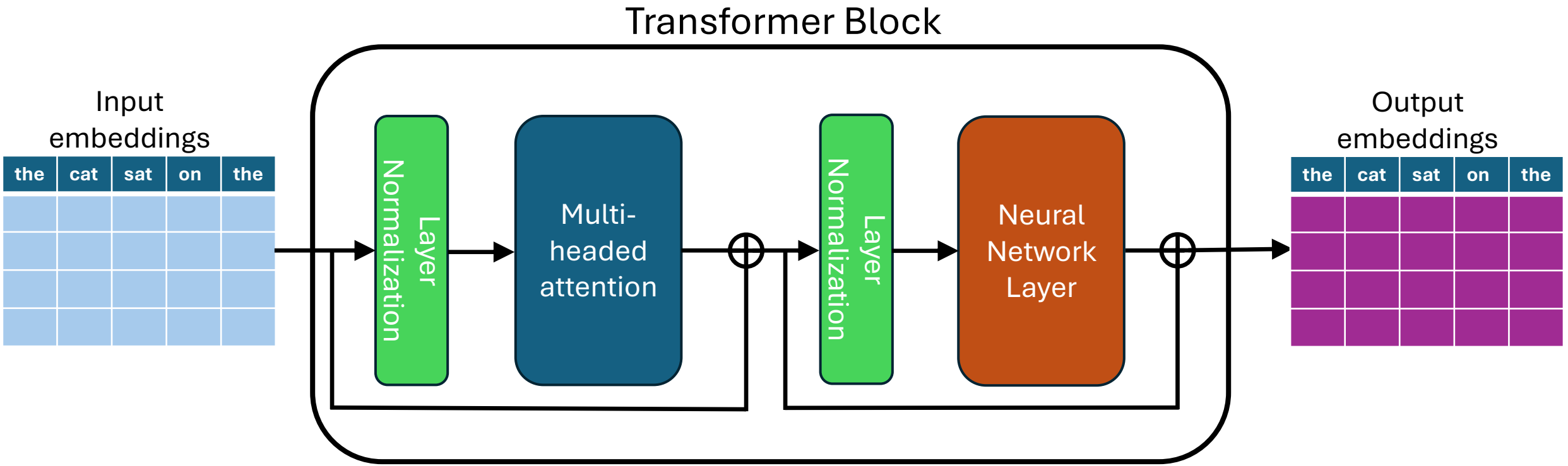
Neural Network Sub-Layer: Mixture of Experts



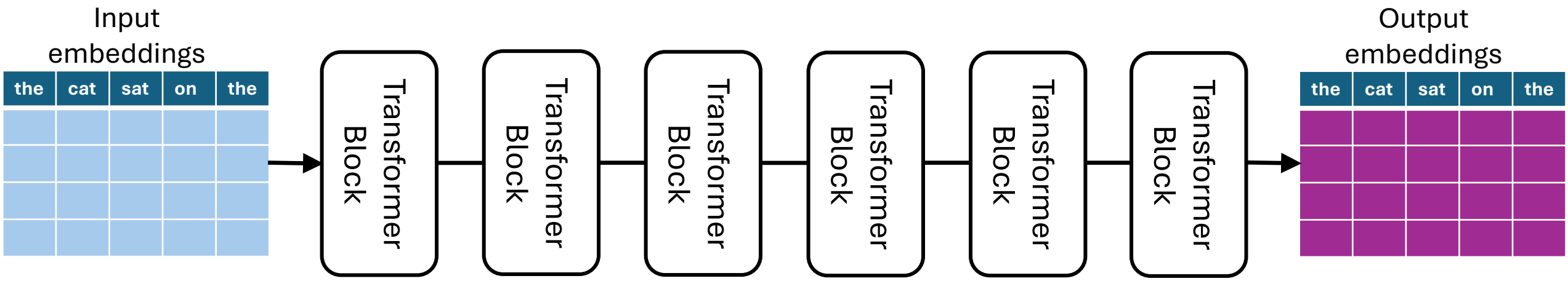
The Transformer Block



The Transformer Block



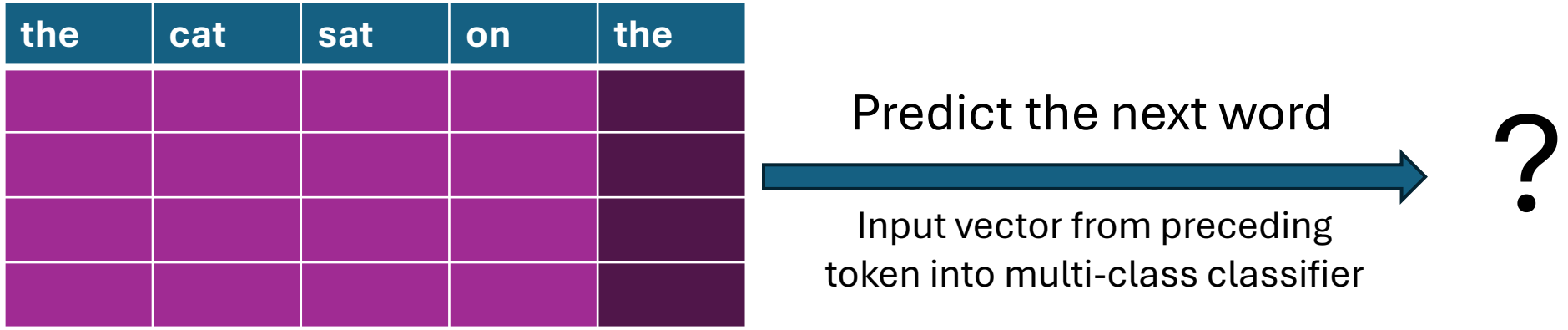
The Transformer Model



Back to predicting the next word

Back to our goal

Predicting the next word from context embeddings

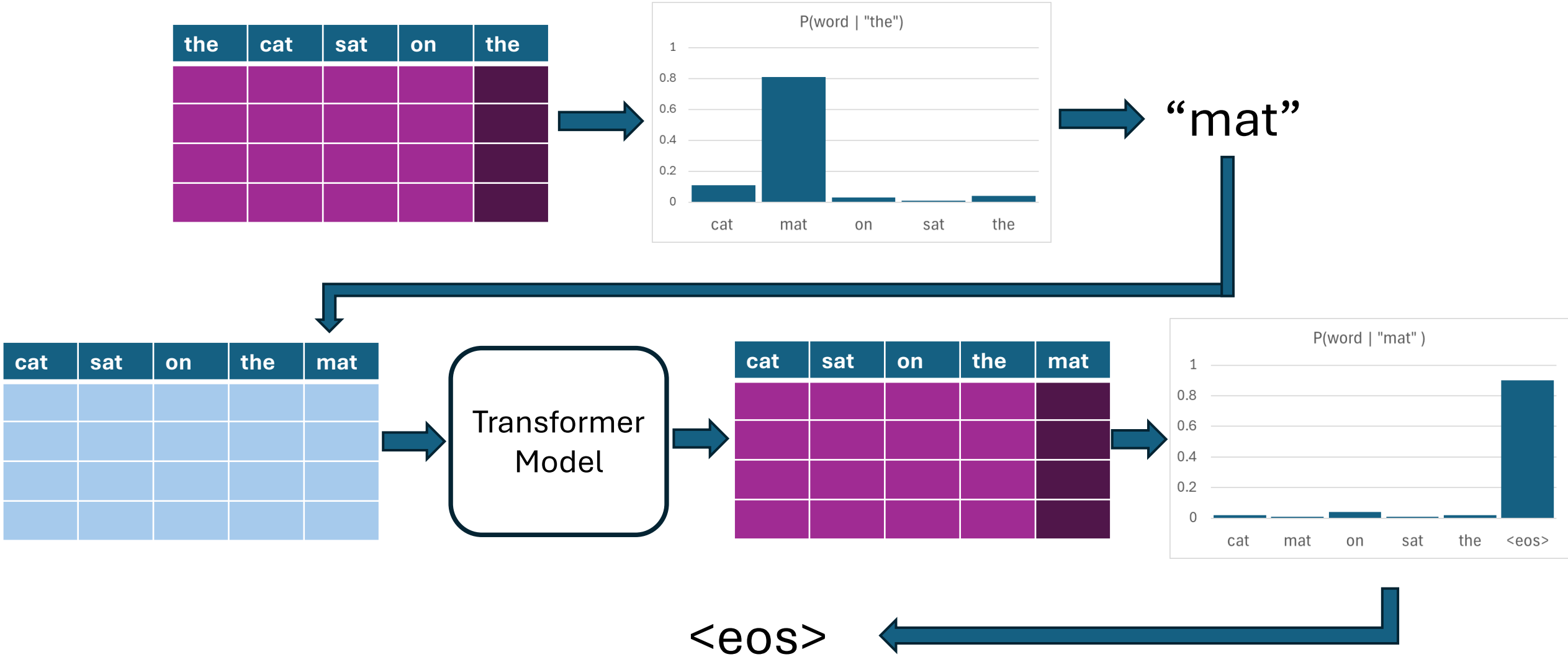


- Use softmax to compute probabilities of each word in our vocabulary given the context embedding vector $\underline{f}$

- $P\left(\text{word } k \mid \text{context embedding vector } \underline{f}\right) = \dfrac{\exp\left(\underline{g_k}^{\mathsf{T}} \underline{f}\right)}{\Sigma_{k'} \exp\left(\underline{g_{k'}}^{\mathsf{T}} \underline{f}\right)}$

- We learn the vectors $\underline{g_k}$ as part of the model training

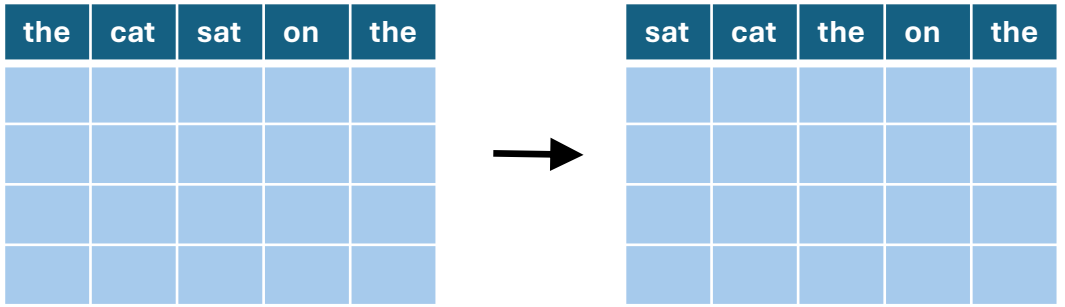
Predicting the next word from context embeddings



Positional encoding

- Let's look at our formula for the attention probabilities

$$A_{n,n'} = \frac{\exp\left(\underline{\text{query}}_n \cdot \underline{\text{key}}_{n'}\right)}{\sum_{n''} \exp\left(\underline{\text{query}}_n \cdot \underline{\text{key}}_{n''}\right)}$$

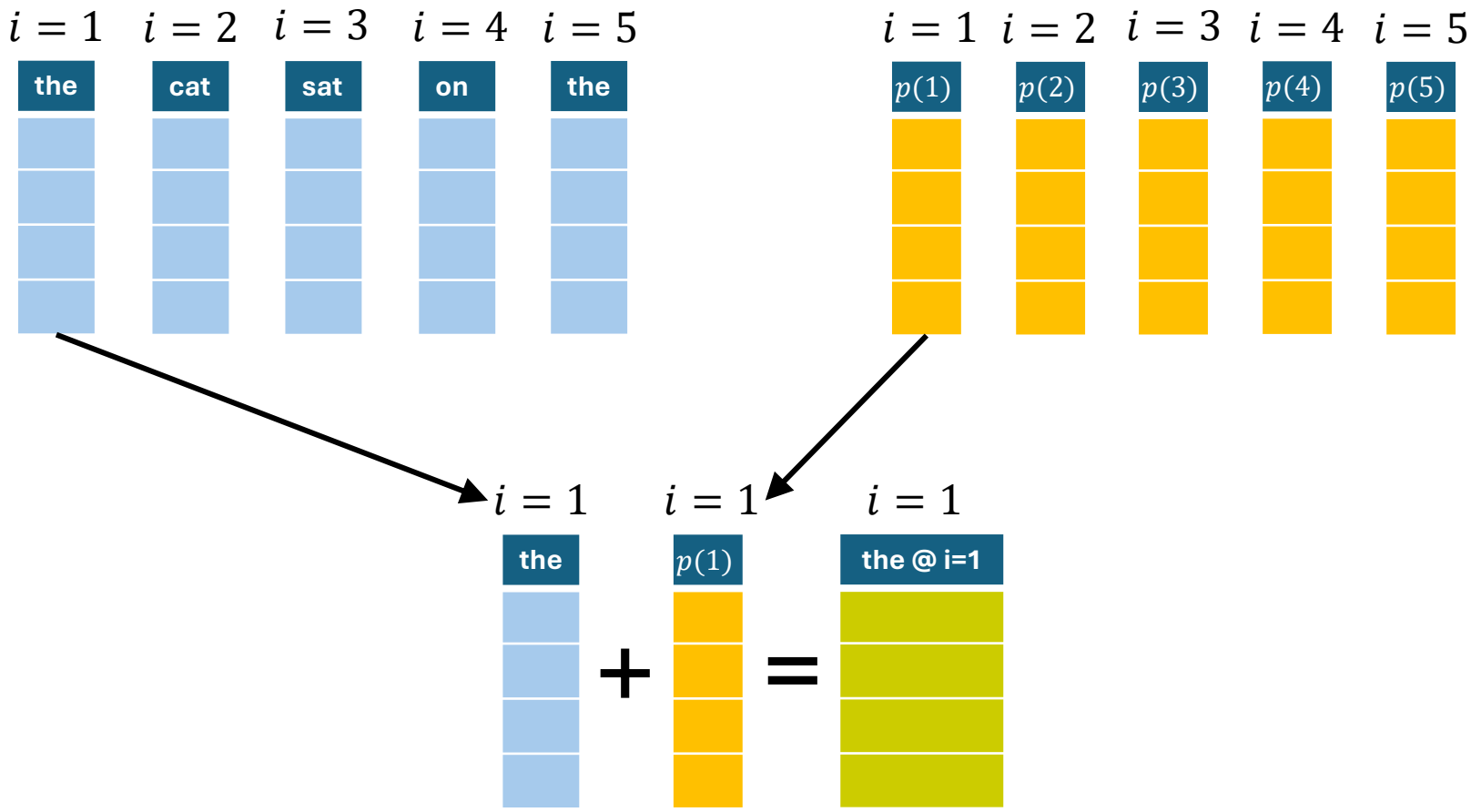


$A_{\text{on,sat}}$ is unchanged

The input embeddings, and hence the context embeddings don't take into account the position of the token in the context (other than through masking) !!

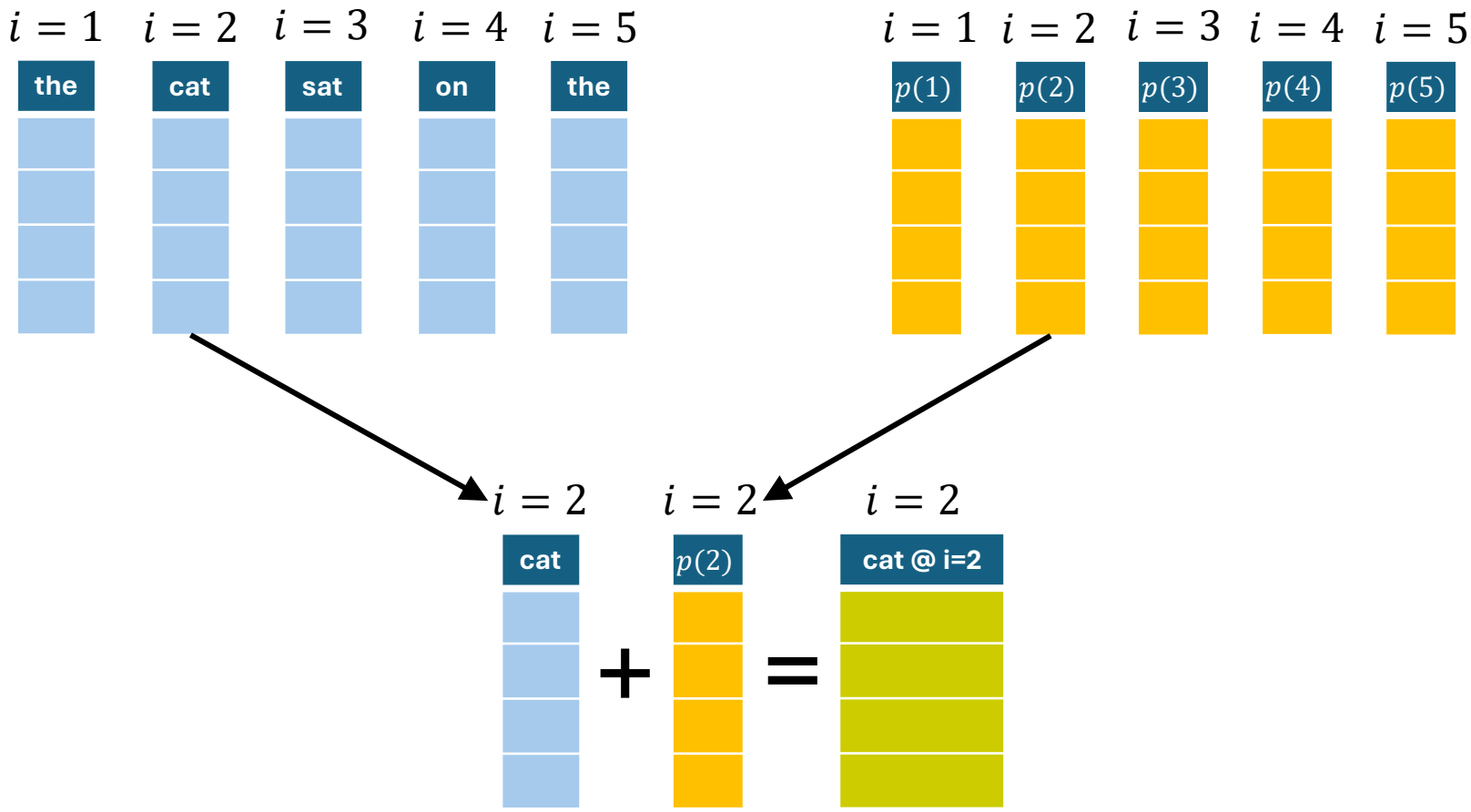
Positional encoding

- We solve by adding position dependent vectors to the input



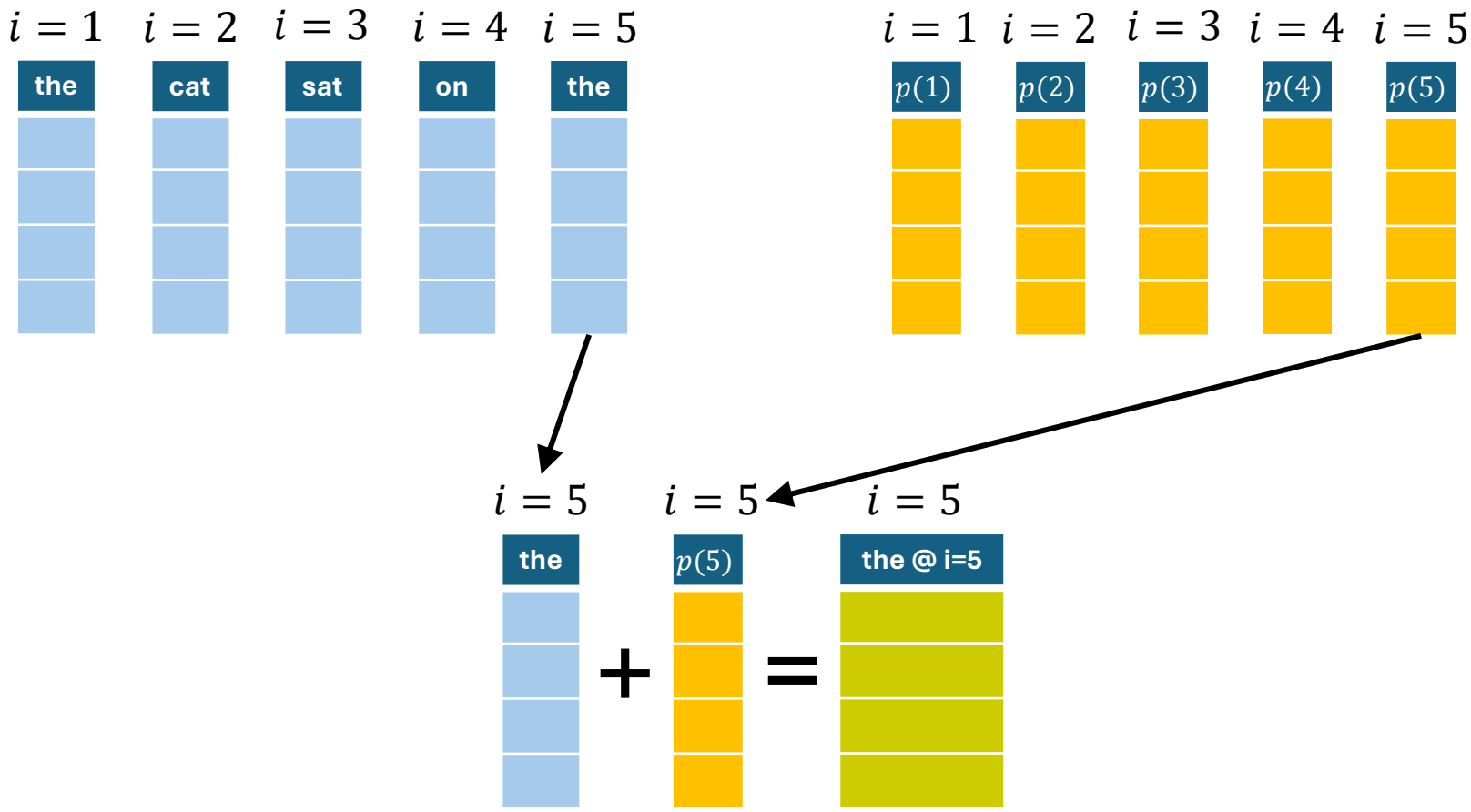
Positional encoding

- We solve by adding position dependent vectors to the input



Positional encoding

- We solve by adding position dependent vectors to the input



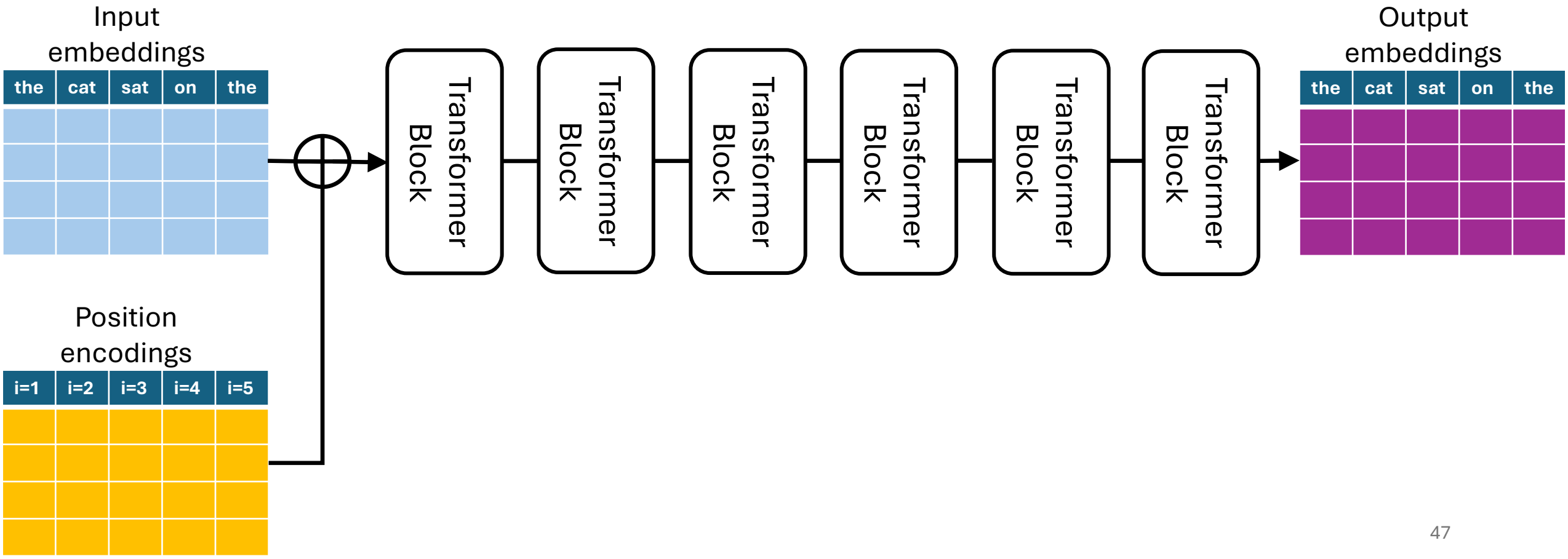
Positional encoding

- Example functions
  - $p_{2j}(i) = \sin\left(\frac{i}{10000^{2j/d}}\right)$
  - $p_{2j+1}(i) = \cos\left(\frac{i}{10000^{2j/d}}\right)$

$$j = 1,2,\ldots,\lfloor d/2 \rfloor$$

Positional encoding

- Putting it all together (finally!)

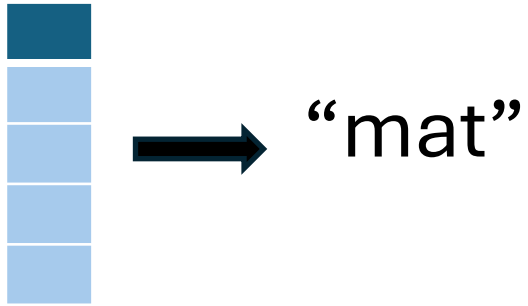


Different types of transformers

Encoders, Decoders, Cross-attention

Different types of transformers

- What we covered was embedding vector → token (word)

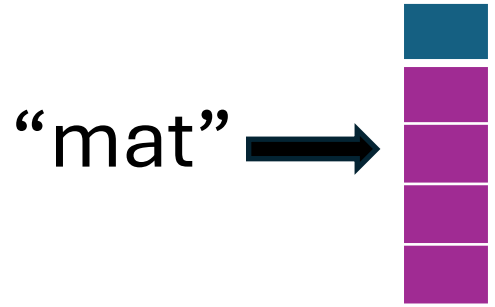


- We ‘decoded’ the embedding vector

- ChatGPT, Claude, etc are known as ‘decoder-only’ models

Different types of transformers

- We also have encoders: token → context embedding vector



- Encoders are useful where we have all the tokens (no prediction), we just need to map them to appropriate vectors

- E.g. “the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

Decoder model, with masking

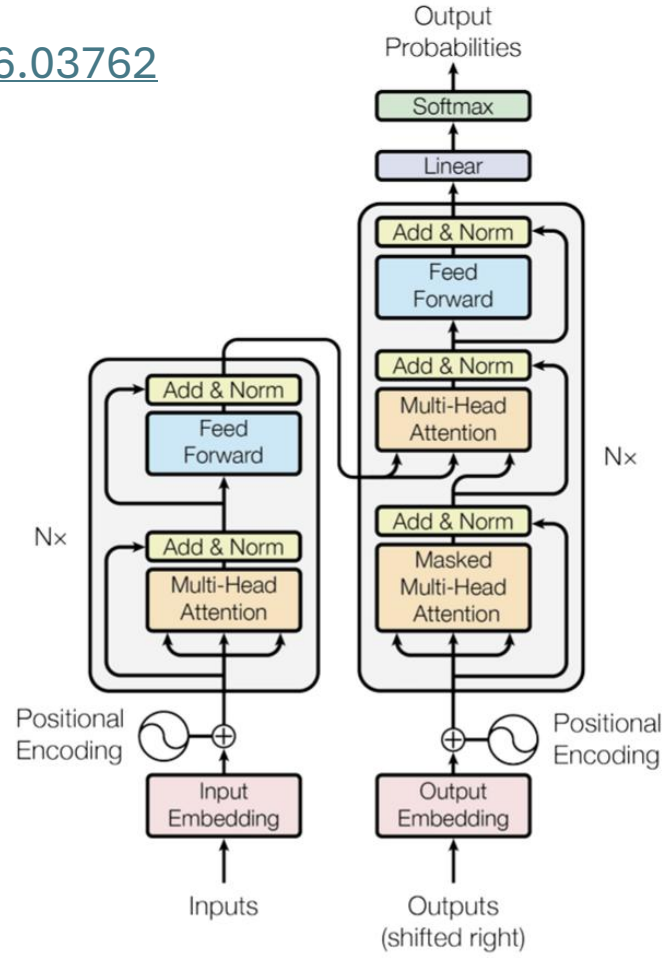
Different types of transformers

‘Attention is all you need’, Vaswani *et al*, 2017 , arXiv:1706.03762

“the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

Decoder model, with masking

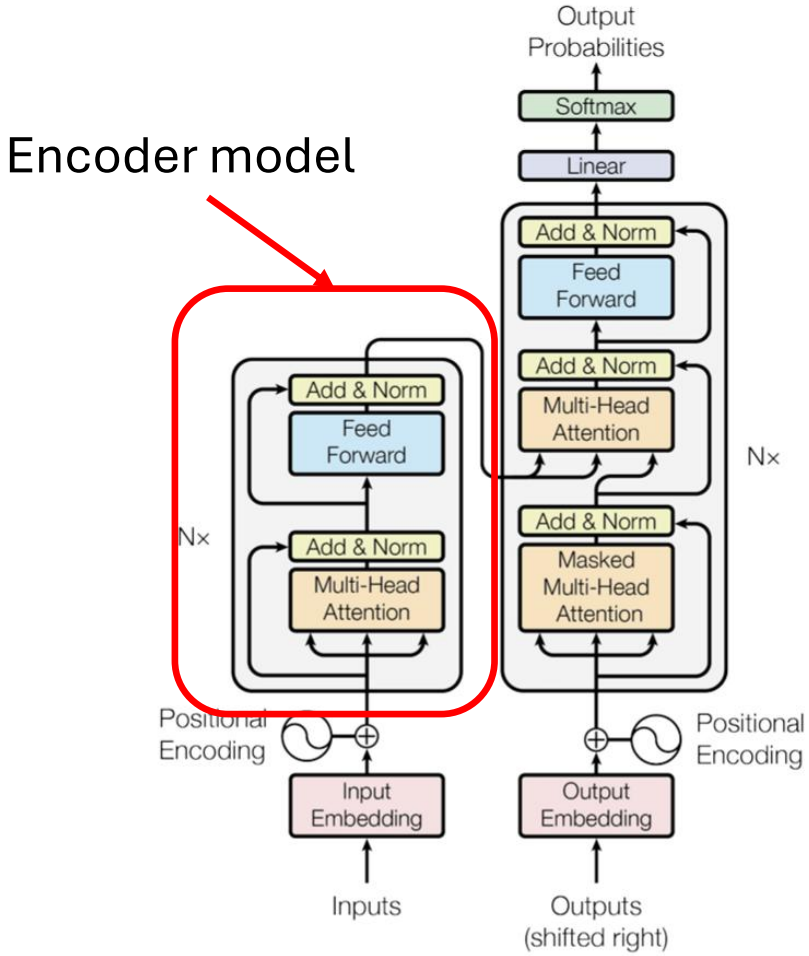


Different types of transformers

“the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

Decoder model, with masking



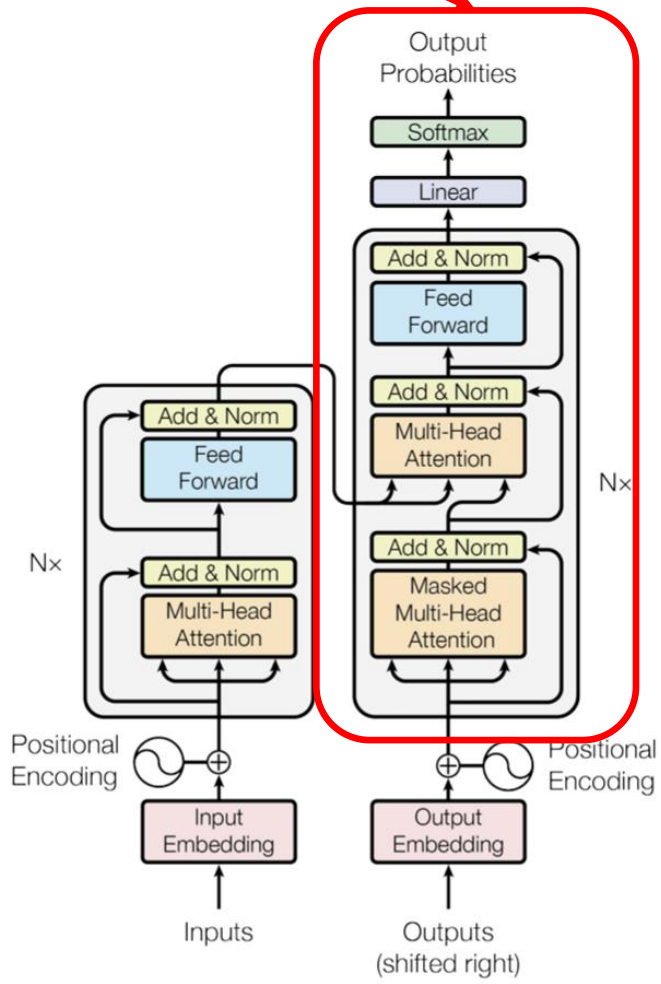
Different types of transformers

“the cat is on the table” → “le chat est sur la table”

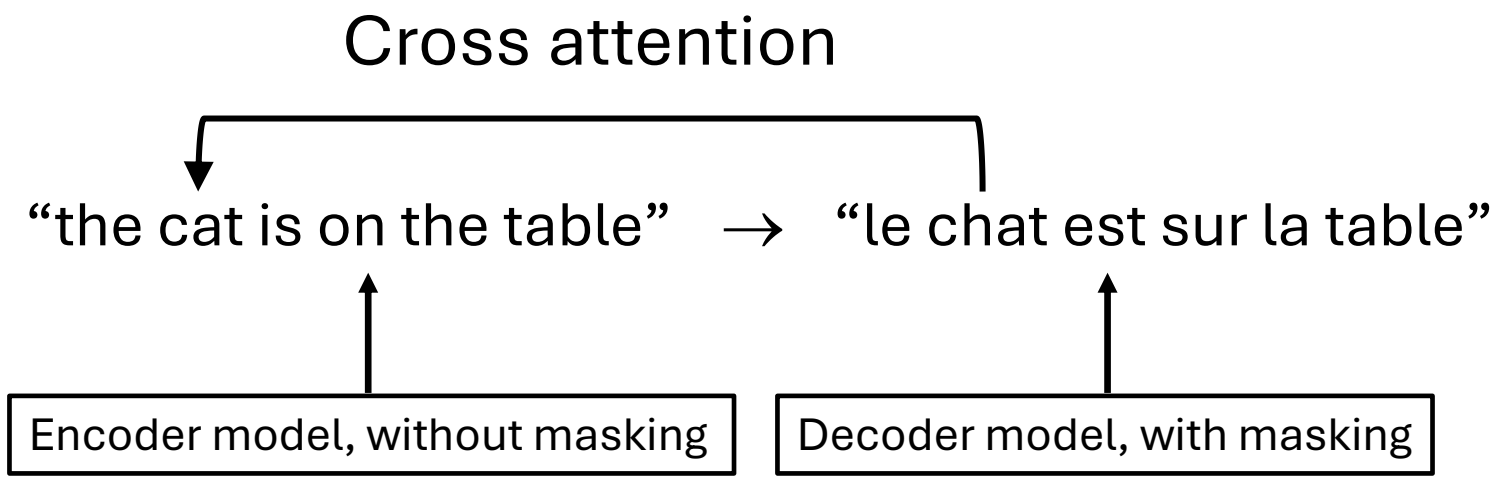
Encoder model, without masking

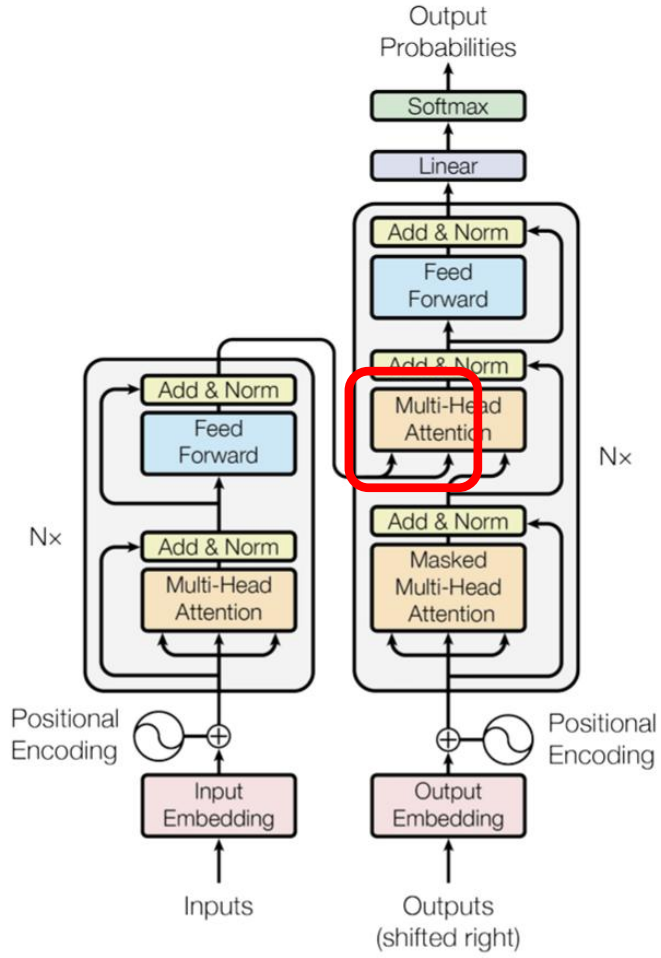
Decoder model, with masking

Decoder model



Different types of transformers





Code example 2

Using PyTorch to build transformer layers

Code Example 2

- Open up the Jupyter notebook ODSCWest2026_CodeExample2.ipynb in the github repository

https://github.com/dchoyle/ODSCWest2026_TransformerBootcamp/

Wrap up: What we learnt

- Next token prediction is a powerful capability

- To solve:
  1) We map very large vocabularies to lower dimensional vectors
  2) Each token in a sequence is initially represented by an embedding vector
  3) We transform each token to a new vector that also encodes the surrounding context

- The transformation model contains lots of parameters.
- We learn the model parameters by training on very large datasets

- Transformer models can be used to encode and/or decode tokens

- We can have applications that use only decoder blocks, only encoder blocks, or both.

Wrap up: What to learn next

- The hundred page language models book - Andriy Burkov

- An introduction to Transformers – R. Turner, arXiv:2304.10557

- Decoder-Only Transformers: The Workhorse of Generative LLMs
  - Cameron R. Wolfe. Substack article

- Attention is all you need – Vaswani et al, arXiv:1706.03762

- Paper Walkthrough: Attention Is All You Need
  - Muhammad Ardi. Towards Data Science article

Wrap up: What I haven't told you

- The engineering challenges, scale, and costs associated with foundational model training are significant

- Foundational models are general – but can be fine-tuned
  - Domain specific datasets
  - LoRA

- Problem of hallucinations can be mitigated (but not eliminated)
  - RAG
  - CoT

Wrap up: Wider reading

- The Hitchhiker's Guide to Agentic AI: From Foundations to Systems
  - Haggai Roitman, arXiv: 2606.24937

- Welcome to the era of experience – David Silver, Richard Sutton
  - Google DeepMind

- The Infinity Machine: Demis Hassabis, DeepMind and the Quest for Superintelligence
  - Sebastian Mallaby

Thank you for listening
Questions?

github.com/dchoyle/ODSCWest2026_TransformerBootcamp

www.linkedin.com/in/davidchoyle/

www.hoyleanalytics.org